

سوالات تحلیلی

۱. تفاوت به کارگیری دستورالعمل LEA با انتقال آدرس به رجیستر از طریق دستورالعمل MOV چیست؟

پاسخ:

- LEA means Load Effective Address
- MOV means Load Value

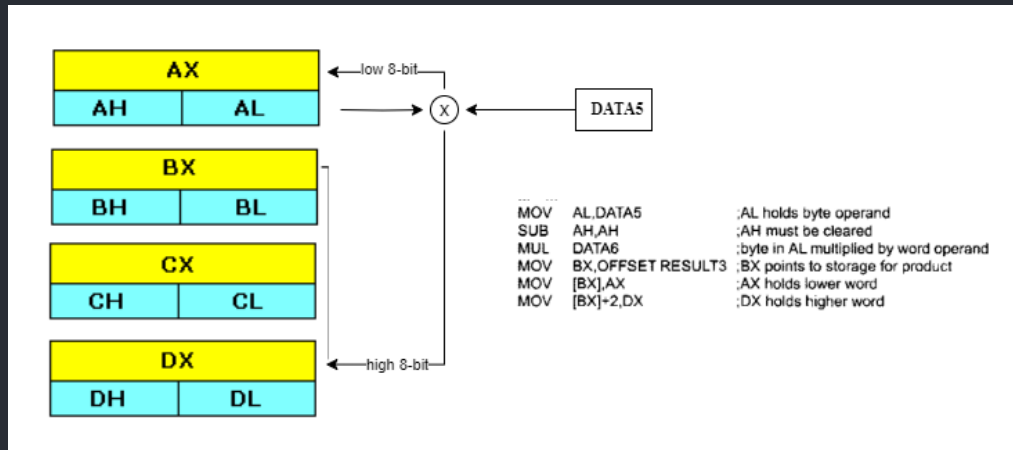
دستور LEA پوینتری از آدرس داده شده را به dest انتقال میدهد؛ اما دستور MOV مقدار آن آدرس را انتقال میدهد.

همچنین در دستور LEA میتوان بر روی آدرس دستکاری انجام داد و سپس آن را ذخیره کرد.

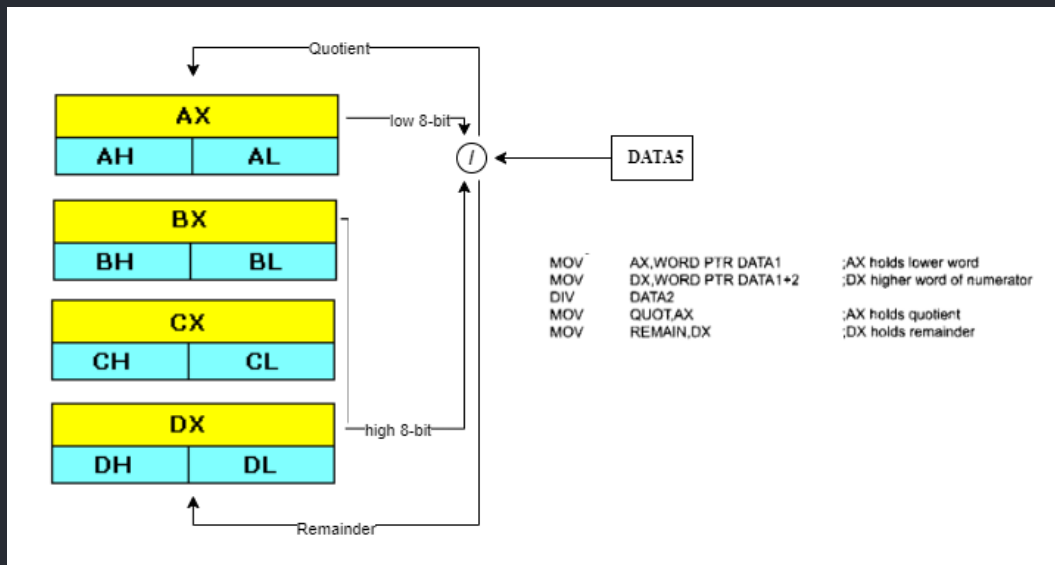
۲. با رسم دیاگرامی نشان دهید برای ضرب ۸-بیت در ۱۶-بیت و نیز تقسیم ۳۲-بیت بر ۱۶-بیت کدام رجیسترها دربردارنده مقادیر ورودی و خروجی هستند؟

پاسخ:

در ضرب word x byte ضرب شونده ۸بیتی در AL قرار دارد. و پاسخ در DX:AX ریخته میشود.



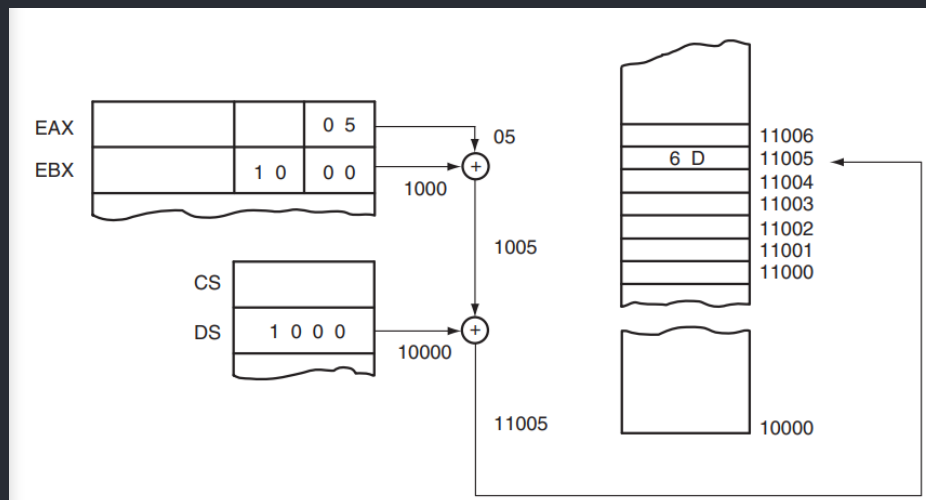
در تقسیم doubleword / word تقسیم شونده 32بیتی در DX:AX و خارج قسمت در AX و باقی مانده در DX قرار میگیرد.



۳. دستور XLAT چه عملکردی دارد و برای تبدیل چه الگویی در زبانهای سطح بالا به زبان اسمبلی استفاده میشود؟

پاسخ: دستور XLAT محتوای رجیستر AL را با بایتی در حافظه عوض میکند. به گونه ای که ابتدا محتوای AL را با BX جمع میکند تا OFFSET یی از مموری را تشکیل دهد. در ادامه محتوای DS:[BX+AL] را در AL کپی میکند.

عملکرد آن مانند INDEXING در آرایه ها است.



۴. با مراجعه به پیوست A مرجع Brey فایل در اختیار گذاشته شده در مراجع درس، بگویید چه دسته سرویس‌هایی از طریق وقفه 21H در اختیار طراح قرار داده شده است؟

پاسخ:

به طور کلی به 5 دسته تقسیم میشوند:

- 1) Terminate Process > 4CH
- 2) ASCII Control Characters > 08H, 09H, 0AH, 0CH, 0DH, 1BH
- 3) Output Functions > 02H, 06H, 05H, 09H, 40H
- 4) Input Functions > 01H, 06H, 0AH, 0BH, 3FH
- 5) Date/Time Functions > 2AH, 2BH, 2CH, 2DH

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN). no echo	08	Char read from STDIN. no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

گزارش کار

روتین ۱:

توضیحات به صورت کامنت در کد ذکر شده است

```

lab1.pdf  Routine2.asm  Main.asm  Routine1.asm X
Micro-Archive > Lab1 > Routine1.asm
1  .MODEL SMALL
2  .STACK 64
3  .DATA
4      TEN DW 000AH
5      n DB ?
6      r DB 01H
7      x_fact DB ? ; IS USED TO CALCULATE factorial of x
8      lf DB 10
9      cr DB 13
10     n_fact DW ?
11     r_fact DW ?
12     n_r_fact DW ?
13     comb DB ? ; ANSWER OF Combination
14     OUTER_LOOP_COUNT_TEMP DW ?
15     STACK_COUNT DB 00H
16     STROUT DB 50 DUP(0)
17 .CODE
18
19 MAIN    PROC FAR
20
21     ; --- Data Segment Define ---
22     MOV AX, @DATA
23     MOV DS, AX
24     MOV DI, OFFSET STROUT
25     ; -----
26
27     get_n:
28     ; --- GETS CHR FROM USER ---
29     MOV AH, 01H
30     INT 21H
31     ; -----
32
33     ; --- ASCII to BCD ---
34     MOV AH, AL
35     AND AH, 0FH
36     ; -----
37     DEC AH
38     MOV n, AH ; SAVE TO n
39
40     CALL PRINT_BR
41
42
43     calculation:
44     MOV CX, 0000H; COUNTER = 0
45     L1:
46     ; MOV BL, n
47     ; CMP CL, BL
48     CMP CL, n
49     JA L1_END ; FINISH WHEN COUNTER > n
50

```

```

50
51     MOV r, CL ; r = COUNTER
52
53     ; --- n! ---
54     MOV AH, n
55     MOV x_fact, AH ; x = n
56     MOV BL, AH
57     CALL FACT
58     SHL DX, 16 ; SHIFT LEFT 16 BITS
59     OR DX, AX ; CONCAT DX AND AX
60     MOV n_fact, DX
61     ; -----
62
63     ; --- r! ---
64     MOV DL, r
65     MOV x_fact, DL ; x = r
66     CALL FACT
67     SHL DX, 16 ; SHIFT LEFT 16 BITS
68     OR DX, AX ; CONCAT DX AND AX
69     MOV r_fact, DX
70     ; -----
71
72     ; --- (n-r)! ---
73     MOV DL, n ; DL = n
74     SUB DL, r ; DL = n - r
75     MOV x_fact, DL ; x = n - r
76     CALL FACT
77     SHL DX, 16 ; SHIFT LEFT 16 BITS
78     OR DX, AX ; CONCAT DX AND AX
79     MOV n_r_fact, DX
80     ; -----
81
82     ; --- n!/(r! (n-r)!) ---
83     MOV AX, n_fact
84     SUB DX, DX
85     DIV r_fact
86     SUB DX, DX
87     DIV n_r_fact
88     MOV comb, AL
89     ; -----
90
91
92     SUB AX, AX ; Clear AX
93     MOV AL, comb ; AX = comb
94
95     convert_to_ascii:
96     SUB DX, DX ; Clear DX for division
97     DIV TEN ; Divide AX by 10
98     ADD DL, '0' ; Convert the remainder to ASCII
99

```

```

100 ; ***** PUSH NUMBERS TO PRINT IT (REVERSE OF REVERSE = ITSELF) *****
101 PUSH DX
102 INC STACK_COUNT
103 ; *****
104
105 CMP AX, 0000H ; CHECKING IF AX IS ZERO
106 JNE convert_to_ascii ; JUMP IF AS IS ZERO
107
108
109
110 save_ascii:
111 ; ***** POP NUMBERS TO PRINT IT (REVERSE OF REVERSE = ITSELF) *****
112 CMP STACK_COUNT, 00H
113 JE save_end
114 POP DX
115 DEC STACK_COUNT
116 ; *****
117
118
119 ; ***** SAVE OUTPUT comb NUMBERS AS ASCII *****
120 MOV [DI], DL
121 INC DI
122 ; *****
123
124 ; --- PRINT char ---
125 MOV BX, AX
126 MOV AH, 02H
127 INT 21H
128 MOV AX, BX
129 ; -----
130
131 JMP save_ascii
132
133 save_end:
134
135 ; --- SAVE AND PRINT SPACE ---
136 MOV [DI], ' ' ; SAVE THE SPACE BETWEEN NUMBERS
137 INC DI
138 MOV DL, 20H ; load ASCII code of space into DL
139 MOV AH, 02H ; select service 02H (print character)
140 INT 21H ; call DOS interrupt
141 ; -----
142
143 INC CX ; COUNTER++
144 INC CX ; COUNTER++
145 LOOP L1 ; COUNTER--
146

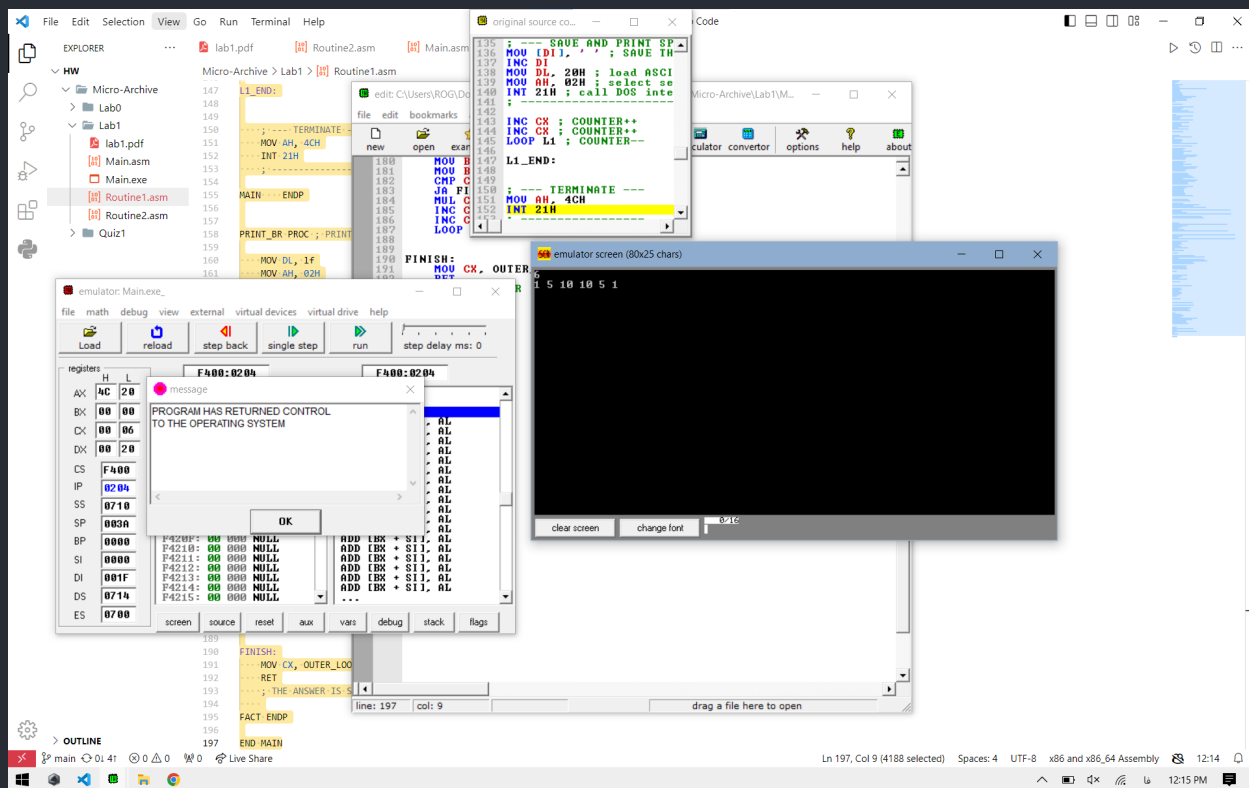
```

```

147  ✓ L1_END:
148
149
150      ; --- TERMINATE ---
151      MOV AH, 4CH
152      INT 21H
153      ; -----
154
155  MAIN      ENDP
156
157
158  ✓ PRINT_BR PROC ; PRINTS NEWLINE
159
160      MOV DL, 1f
161      MOV AH, 02H
162      INT 21H
163      MOV DL, cr
164      MOV AH, 02H
165      INT 21H
166      RET
167
168  PRINT_BR ENDP
169
170
171  ✓ FACT PROC ; Saves x! IN (DX, AX)
172
173      MOV OUTER_LOOP_COUNT_TEMP, CX
174
175      MOV CX, 01H ; COUNTER = 1
176      MOV AX, 01H ; AX = 1
177      MOV DX, 0000H ; CLEAR DX
178
179  ✓ BACK:
180      MOV BH, 00H
181      MOV BL, x_fact
182      CMP CX, BX
183      JA FINISH ; FINISH WHEN COUNTER > x
184      MUL CX ; (DX, AX) = CX * AX
185      INC CX ; COUNTER++
186      INC CX ; COUNTER++
187      LOOP BACK ; COUNTER--
188
189
190  ✓ FINISH:
191      MOV CX, OUTER_LOOP_COUNT_TEMP
192      RET
193      ; THE ANSWER IS STORED IN (DX, AX)
194
195  FACT ENDP
196
197  END MAIN

```

تست و خروجی نمونه:



روتین ۲:

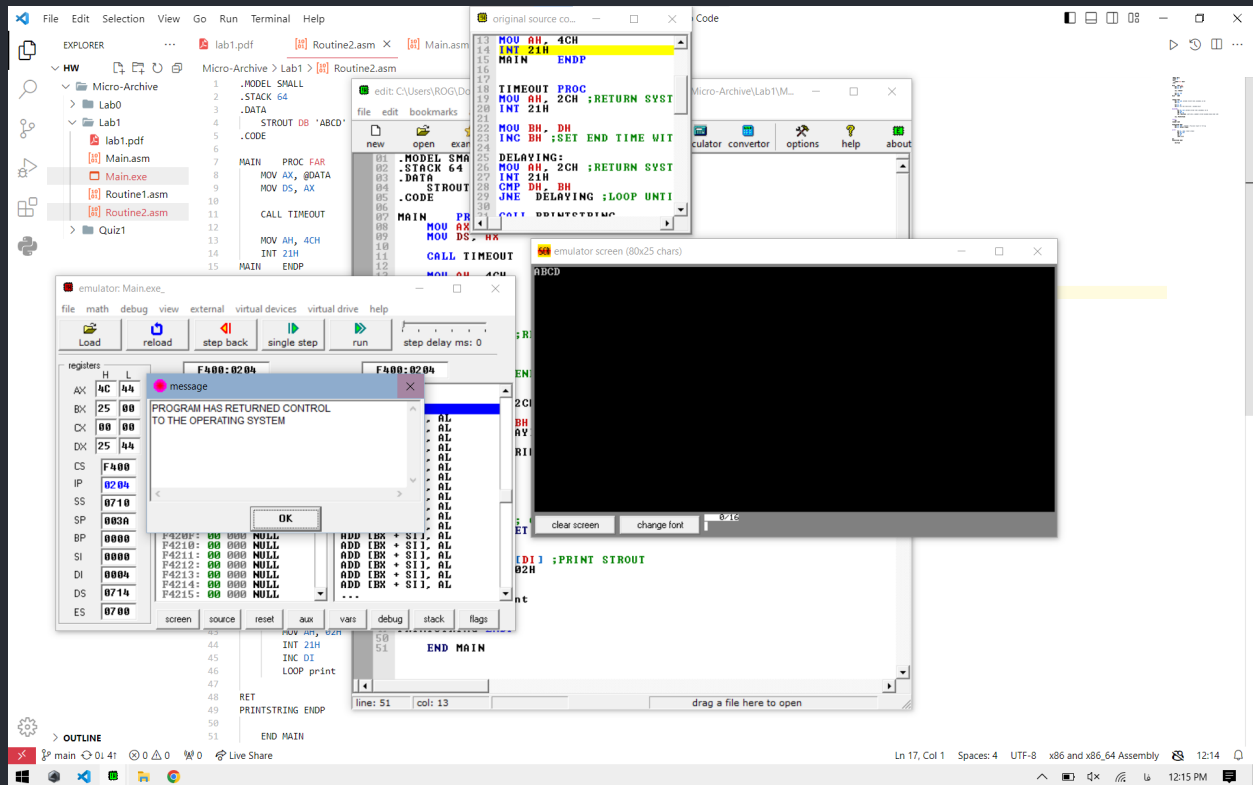
در این روتین یک رشته با Delay یک ثانیه چاپ می‌شود. رشته نمونه ی این روتین "ABCD" می‌باشد.

```

1  .MODEL SMALL
2  .STACK 64
3  .DATA
4  |   STROUT DB 'ABCD'
5  .CODE
6
7  MAIN    PROC FAR
8  |   MOV AX, @DATA
9  |   MOV DS, AX
10
11  |   CALL TIMEOUT
12
13  |   MOV AH, 4CH
14  |   INT 21H
15  MAIN    ENDP
16
17
18  TIMEOUT PROC
19  |   MOV AH, 2CH ;RETURN SYSTEM TIME [SECONDS] IN DH
20  |   INT 21H
21
22  |   MOV BH, DH
23  |   INC BH ;SET END TIME WITH 1 SECOND DELAY
24
25  DELAYING:
26  |   MOV AH, 2CH ;RETURN SYSTEM TIME [SECONDS] IN DH
27  |   INT 21H
28  |   CMP DH, BH
29  |   JNE DELAYING ;LOOP UNTIL CURRENT SYSTEM SECONDS REACH END TIME
30
31  |   CALL PRINTSTRING
32
33  return:
34  |   RET
35  TIMEOUT ENDP
36
37  PRINTSTRING PROC
38  |   MOV CX, 04H ; COUNTER = Maximum length of String
39  |   MOV DI, OFFSET STROUT
40
41  print:
42  |   MOV DL, [DI] ;PRINT STROUT
43  |   MOV AH, 02H
44  |   INT 21H
45  |   INC DI
46  |   LOOP print
47
48  RET
49  PRINTSTRING ENDP
50
51  END MAIN

```

تست و خروجی نمونه:



برنامه اصلی (خیام پاسکال کامل):

برنامه Main.asm، ترکیب دو روتین قبلی هست که با گرفتن عدد n ، خط اول مثلث خیام را با یک ثانیه تاخیر در بین هر خط چاپ می‌کند.

خروجی:

